

Jetson 端工业缺陷检测的输入数据路径重构

王凯伦, 王琦

合肥工业大学数学学院, 安徽 合肥 230601

通信作者: 王琦

摘要: 针对边缘工业缺陷检测中网络低精度化不能直接消除输入形成与主机—设备数据移动开销的问题, 本文将固定推理对象下的输入数据路径抽象为跨边界表示、输入张量形成位置、额外打包原始图像暂存策略和执行拓扑四类结构变量。在固定 YOLOv8n、TensorRT INT8 混合精度 Engine 和单帧顺序拓扑的条件下, 构造包含跨边界表示、张量形成位置及相应暂存组织变化的 $V0 \rightarrow V2R$ 路径级重构, 以及 $V2R \rightarrow V3R$ 暂存策略级细化两级受控干预, 并以任务正确性保持为性能比较约束。冻结的 180 幅图像离线回放实验表明, 三条路径的任务级指标一致; $V2R$ 相对 $V0$ 达到 $2.24 \times$ FPS, 平均端到端延迟降低 55.45%; $V3R$ 相对 $V2R$ 的 FPS 提高 4.07%, 平均延迟降低 4.03%, 而 P95 增加 0.15%、P99 降低 0.12%, 尾延迟变化方向相反。结果说明, 在受测平台与配置下, 完整路径级重构对应较大的平均响应, 局部暂存策略变化仅产生有限平均增量, 且未形成一致的尾延迟改善证据。对于本文这类固定推理对象的数据路径评价, 应区分输入表示、张量形成位置和主机暂存策略, 并分别考察平均与尾部响应。

关键词: Jetson; 工业缺陷检测; 输入数据路径; 端到端推理; 受控比较

Input Data-Path Reconstruction for Industrial Defect Detection on Jetson

WANG Kailun, WANG Qi

School of Mathematics, Hefei University of Technology, Hefei 230601, China

Corresponding author: WANG Qi

Abstract: For edge-based industrial defect detection, reducing network precision does not by itself eliminate input formation and host-device data-movement costs. This study abstracts the input data path of a fixed inference object by four structural variables: the representation crossing the host-device boundary, the location where the input tensor is formed, the policy for additional packed raw-image host staging, and the execution topology. With a fixed YOLOv8n detector, TensorRT INT8 mixed-precision Engine, and sequential single-frame topology, two hierarchical controlled interventions are constructed: the $V0$ -to- $V2R$ path-level reconstruction, which changes the representation, tensor-formation location, and corresponding staging organization, and the $V2R$ -to- $V3R$ staging-policy refinement. Performance comparisons are admitted only under preserved task correctness. Offline replay of the frozen 180-image workload yields identical task-level metrics for all three paths. Relative to $V0$, $V2R$ achieves $2.24 \times$ FPS and 55.45% lower mean end-to-end latency. Relative to $V2R$, $V3R$ improves FPS by 4.07% and lowers mean latency by 4.03%, whereas P95 increases by 0.15% and P99 decreases by 0.12%, giving opposite tail-latency directions. Under the tested platform and configuration, the complete path-level reconstruction corresponds to a substantial mean response, while the local staging-policy change yields only a limited mean increment and no consistent tail-latency improvement. For fixed-inference-object data-path evaluations of this type, input representation, tensor-formation location, and host staging policy should be distinguished, with mean and tail responses considered separately.

Keywords: Jetson; industrial defect detection; input data path; end-to-end inference; controlled comparison

中图分类号: TP391.41

0 引言

金属表面缺陷会影响产品质量与后续制造过程, 因而需要在生产现场及时完成视觉检测。NEU-DET 覆盖开裂、夹杂、斑块、点蚀表面、轧制氧化皮和划痕等典型热轧钢带缺陷, 为这一任务提供了公开研究对象^[1,2]。当检测过程部署到靠近数据源的边缘设备时, 评价目标不仅是模型精度, 还包括固定任务语义下的完整处理响应。

围绕检测模型本身, 已有研究通过轻量化骨干、注意力机制和多尺度特征融合改善精度与复杂度权衡^[3-5], YOLOv8 则提供多尺度检测模型及训练、推理和导出链路^[6]。这类工作主要回答“采用何种检测器及网络结构”, 为边缘部署提供模型基础, 但不能单独描述模型执行前后的系统数据组织。

另一类工作把研究范围扩展到边缘推理运行时。完整检测链通常同时包含数据获取、预处理、

模型执行、后处理与数据管理^[7]；CPU 与 GPU 之间的预处理调度会改变运行时路径^[8]，工业现场应用也要求在设备附近完成端到端处理^[9]。因此，仅报告网络推理时间不足以刻画输入形成方式变化对完整系统响应的影响。

模型低精度化进一步降低网络侧计算与存储开销，但量化误差仍须受到任务正确性约束^[10,11]。本文固定采用 TensorRT 10.3 校准式 INT8 混合精度 Engine；该版本的传统隐式 INT8 量化与 calibrator 接口具有明确的软件版本边界^[12]。量化部署研究同样表明，性能比较需要同时维持任务行为^[13]。然而，网络精度降低并不会自动决定图像以何种表示跨越主机—设备边界，也不会决定输入张量在主机还是设备侧形成。

在 Jetson 检测部署中，预处理、推理与后处理可形成不同的端到端组织^[14]。CUDA 文档指出主机内存类型与异步复制具有特定适用条件^[15]；集成 CPU-GPU 系统和 GPU 内存分配研究也说明内存策略的响应依赖平台、工作负载与访问方式^[16,17]，其执行语义还受流与复制规则约束^[18]。已有完整边缘处理研究关注并发或流水化组织^[19]。在本文考察的相关边缘部署研究中，现有工作多分别关注模型优化、预处理或主机—设备内存行为，对跨边界表示、输入张量形成位置和主机暂存策略在统一 E2E 路径中的联合组织关系讨论相对有限。

为此，本文将输入数据路径定义为固定推理对象下的结构描述，并以任务正确性保持为比较准入条件。考虑到平均响应与尾部响应反映不同性能维度^[20]，同时结合 Jetson 平台比较的配置依赖性^[21]及缺陷检测基准对数据和评价边界的要求^[22]，本文提出两个研究问题：路径级重构对完整端到端平均响应有何影响；在其余结构变量固定时，暂存策略变化是否产生额外平均响应并带来同方向的 P95/P99 变化。

本文的主要贡献包括两点：1) 面向固定推理对象，建立由跨主机—设备边界表示、输入张量形成位置、额外打包原始图像暂存策略和执行拓扑构成的输入数据路径抽象，并据此形成 V0、V2R 和 V3R 的层级受控重构关系；2) 在任务正确性保持条件下，分别评价路径级重构和暂存策略级细化，联合给出 E2E 均值、吞吐率及 P95/P99 响应，并在受测平台与配置边界内解释两级干预的不同响应。

1 输入数据路径模型与问题表述

1.1 固定推理对象与系统边界

本文将检测器、Engine、输入尺寸、工作负载与后处理语义的组合记为固定推理对象。具体对象为 YOLOv8n、640×640 输入、batch size 1 和同一 TensorRT INT8 混合精度 Engine；该 Engine 启用 INT8 与 FP16 fallback 并保持 FP32 I/O。工作负载固定为 NEU-DET split-v2 的 180 幅测试图像，置信度阈值、IoU 阈值、候选框上限和 class-aware 单标签后处理在全部路径中不变。平台与软件版本属于实验条件，列于表 2，不作为路径结构变量。

系统边界采用单帧顺序执行：从获取数据源帧前开始，经图像解码、输入形成、主机—设备数据移动、Engine 推理与同步、输出回传、CPU 后处理和结果构造，至结果序列化及写文件前结束。该边界不包含跨帧流水线、显式传输/计算重叠、多流执行、zero-copy 或 GPU NMS。

1.2 路径描述符与名义复制载荷

在固定推理对象内，将一条输入数据路径表示为

$$P=(R,F,M,E),$$

式中， R 表示跨越主机—设备边界的数据表示， F 表示 TensorRT 输入张量的形成位置， M 表示额外打包原始图像的主机暂存策略， E 表示执行拓扑。预处理位于 CPU 或 GPU 是 F 的实现映射，而非额外引入的独立控制变量。描述符用于检查各次比较中哪些结构变量发生变化，不用于分解各变量的延迟贡献。

由 R 可推导每帧名义输入复制载荷

$$B(P)=H_R(P)W_R(P)C_R(P)S_R(P).$$

其中， H_R, W_R, C_R, S_R 分别为跨边界表示的高、宽、通道数与单元字节数。实例化后， $B(P_0)=640 \times 640 \times 3 \times 4 = 4.9152 \text{ MB/frame}$
 $B(P_2)=B(P_3)=200 \times 200 \times 3 \times 1 = 0.1200 \text{ MB/frame}$
 因而 $B(P_0)/B(P_2)=40.96$ 。该比值是由表示结构得到的名义载荷对比，不是实测总线或 DRAM 流量、带宽、H2D 时间或传输加速比。

表 1 将描述符实例化。 P_0 对应 V0： R 为 FP32 NCHW 张量， F 在主机侧， M 无额外打包原始图像暂存； P_2 对应 V2R： R 为 packed BGR uint8， F 在设备侧， M 为 pageable； P_3 对应

V3R，其余变量与 P_2 相同，仅 M 改为 pinned。三者的 E 均为单帧顺序执行。

表 1 三条输入数据路径的结构描述与派生量。名义输入复制载荷由跨边界表示推导，非实测流量。

路径描述项	V0 (P_0)	V2R (P_2)	V3R (P_3)
H2D 数据表示 R	FP32 NCHW 张量	packed BGR uint8	packed BGR uint8
输入张量形成位置 F	主机	设备	设备
预处理位置	CPU	GPU 融合处理	GPU 融合处理
额外打包原始图像暂存策略 M	无	Pageable	Pinned
名义输入复制载荷 $B(P)$	4.9152 MB/frame	0.1200 MB/frame	0.1200 MB/frame
执行拓扑 E	单帧顺序	单帧顺序	单帧顺序

1.3 层级受控比较、正确性条件与评价问题

如图 1 所示， $P_0 \rightarrow P_2$ 同时改变跨边界表示和张量形成位置，并引入相应的原始图像暂存，即 R, F, M 发生变化而 E 保持不变，属于路径级

重构； $P_2 \rightarrow P_3$ 只改变 M ，属于局部暂存策略细化。这里的层级仅表示受控变量范围不同，不预设收益大小，也不构成阶段级因果分解。

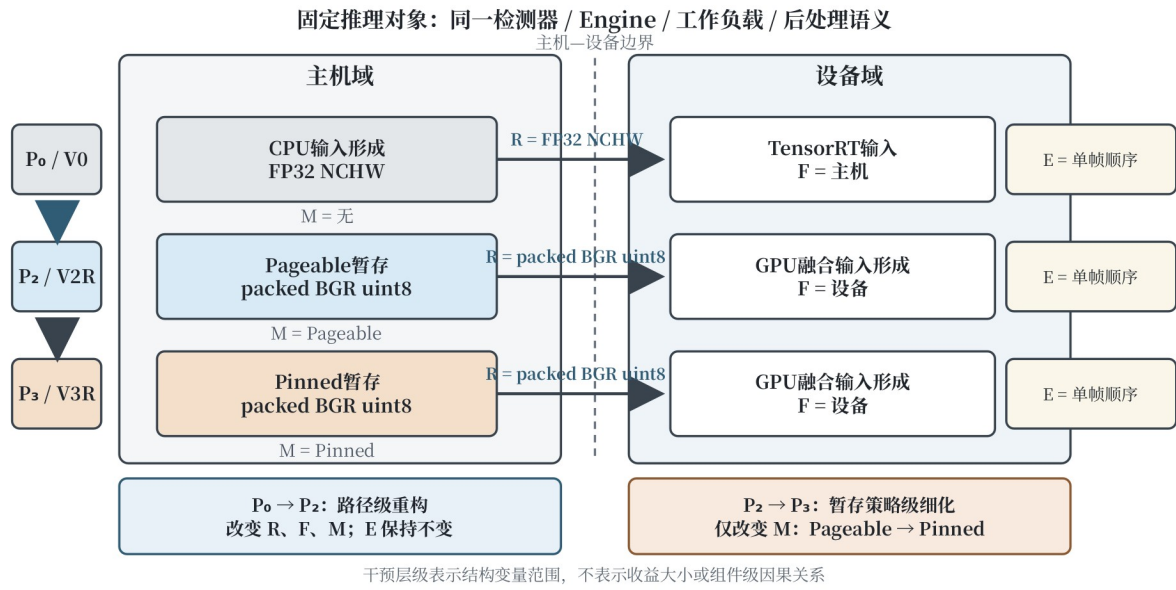


图 1 输入数据路径抽象及层级受控比较。图中层级表示结构变量的干预范围，不表示收益大小或组件级因果关系。

性能比较还必须满足正确性条件：三条路径使用同一工作负载、评价器、模型/Engine、输入和后处理语义；V2R 通过预定义的任务级核验，V3R 由同一评价器确定性重算；汇总任务指标一致，类别级 AP50 与 Recall 最大路径间差异为 0。该条件只保证冻结工作负载和评价口径内的可比性，不表示逐位相等或对未来输入普适等价。

完整路径的直接测量响应定义为

$$T_{E2E}(P) = t_{pre-sink}(P) - t_{source}(P),$$

其中， t_{source} 位于获取数据源帧前， $t_{pre-sink}$ 位于结果构造完成且序列化和写文件前。本文评价完整路径 P 的 E2E 响应，不独立测量 H2D、预处理或 CUDA 核函数时间。

据此提出：RQ1，在固定推理对象、工作负载和任务语义时， $P_0 \rightarrow P_2$ 路径级重构对完整 E2E 平均响应产生何种影响？RQ2，在 R, F, E 、

GPU 预处理语义及下游结构保持不变时， $P_2 \rightarrow P_3$ 只改变主机暂存策略，是否产生额外平均响应，以及 P95/P99 是否呈现一致改善？RQ1 由 4.2 节回答，RQ2 由 4.3 和 4.4 节回答。

2 受控输入数据路径重构

2.1 V0 基线路径

V0 实现 P_0 的主机张量路径。数据源解码为 BGR 图像后，CPU/OpenCV 采用 INTER_LINEAR 完成 640×640 letterbox 与常数 114 填充，并完成 BGR → RGB、HWC → CHW 及 1/255 归一化，在主机侧形成 1×3×640×640 FP32 NCHW 张量后执行 FP32 H2D 复制。其科学语义是“主机形成模型输入、张量表示跨越边界”，具体操作仅用于复现该路径实例。

2.2 V2R 路径级重构

V2R 实现 P_2 ，将跨边界表示由 FP32 NCHW 张量改为 packed BGR uint8，并将输入张量形成位置由主机移至设备。冻结工作负载的源图像为 200×200，因此主机侧只额外形成连续的 600 B 行宽 packed BGR pageable 暂存；该表示经二维 H2D 复制后，由设备侧融合预处理直接写入 TensorRT 管理的 FP32 NCHW 输入。实现中二维复制映射为 cudaMemcpy2DAsync，融合处理完成 resize、padding、BGR → RGB、归一化与布局变换。

GPU resize 按 V0 的 OpenCV 4.5.4 INTER_LINEAR 语义建立受控对齐，letterbox 几何和填充值保持一致。该约束服务于三条路径的输入语义一致性，不构成通用 CUDA/OpenCV 等价性声明。相关设备缓冲区和执行上下文在进程内复用，以避免逐帧重复分配或创建。

2.3 V3R 暂存策略细化

V3R 实现 P_3 ，在 V2R 基础上只将额外打包原始图像的主机暂存策略由 pageable 改为 pinned。跨边界表示、设备侧张量形成、GPU 融合预处理、二维复制拓扑、CUDA stream、Engine 和全部下游处理均保持不变。实现上使用 cudaHostAlloc 建立跨帧复用的 pinned 暂存；其分配方式只是 M 的实现映射，不改变复制内容和每帧有效几何。

2.4 共同控制与正确性约束

三条路径共用同一 Engine、CUDA stream 语义、推理同步、输出回传以及 CPU 检测框解码、置信度筛选和 NMS，执行拓扑均为单帧顺序。后处理配置固定为置信度阈值 0.25、IoU 阈值 0.45、max_nms=30000、max_det=300 和 class-aware 单标签语义。V2R 通过预定义任务级核验；V3R 由相同评价程序对冻结预测结果确定性重算，并通过输入顺序、处理数量、丢帧和 EOS 检查确保执行完整性及路径间可比性。

3 实验协议

3.1 实验平台与模型配置

平台、固定推理对象和统一协议见表 2。软件版本、功耗模式和工作负载是复现实验的条件，不参与路径描述符定义。

表 2 平台、模型与统一基准协议。

项目	设置
平台	NVIDIA Jetson Orin Nano Engineering Reference Developer Kit Super
软件栈	L4T R36.5; CUDA 12.6; TensorRT 10.3; OpenCV 4.5.4
Detector / 输入 Engine	YOLOv8n; 640 × 640; batch 1 TensorRT INT8 混合精度 (INT8 + FP16 fallback); Engine 输入张量: FP32
校准	1260 张去重训练图像; Int8EntropyCalibrator2; batch 1; 排除 test split
工作负载	固定 180 张 test 图像
路径	V0 / V2R / V3R; 单帧顺序执行
计时协议	60 帧预热; 每进程 1080 帧; 每路径 5 个独立进程
正式计时	关闭 diagnostics 与 profiling

正式运行采用 MAXN_SUPER 电源模式 (nvpmodel mode 2)，未调用 jetson_clocks，频率未独立归档。运行前温度约 46.8–47.1 °C，运行后约 48.7–49.6 °C；这些非连续观察不用于判断持续温度、运行时降频或功耗状态。

3.2 运行与正确性协议

性能实验使用同一 manifest 限定的 split-v2 固定 180 幅测试图像及顺序。每个独立进程预热 60 帧，同步并重置测量窗口后测量 1080 帧；每条路径执行 5 个独立进程，形成 5400 个逐帧延迟样本，三条路径共 15 个进程和 16200 个样本，路径间不构造运行配对。

15 个独立进程按预先设定的交错顺序执行。全部进程均处理 1080 个测量帧、丢帧为 0，且输入顺序与 EOS 检查通过，确保执行完整性和比较条件一致；逐帧 diagnostics 与 profiling 保持关闭。

3.3 E2E、FPS 与尾延迟指标

正确性采用冻结测试集和统一评价器，报告 Precision、Recall、mAP50、mAP50-95 及类别级 AP50/Recall。性能指标沿用式 (3) 的 source-to-pre-sink 边界；第 i 个独立进程的 FPS 按 $f_i = N / T_i$ 计算，其中 $N=1080$ ， T_i 为完整测量阶段 wall time，故 FPS 不是逐帧延迟的倒数。每条路径报告 5 个进程级 FPS 的均值与样本标准差，并合并 5400 个逐帧样本计算平均延迟、P95 和 P99；百分位采用 Type-7 线性插值。所有统计均为描述性结果，不进行置信区间、假设检验或统计显著性推断。

4 结果与分析

4.1 正确性约束验证

性能响应只有在任务行为保持时才具有路径比较意义。三条路径在冻结 180 幅测试图像和统一评价程序下获得相同的汇总任务指标，类别级最大 AP50 与 Recall 路径间差异均为 0，结果见表 3。V2R 通过预定义任务级核验，V3R 由同一评价器对冻结预测结果确定性重算；因此后续比较处于同一冻结任务行为约束内，但不扩展为逐位相等或未来输入上的普适结论。

表 3 三条路径在冻结工作负载和统一评价程序下的任务级

Path	Precision / Recall / mAP50 / mAP50-95
V0	0.6913 / 0.6991 / 0.6476 / 0.3523
V2R	0.6913 / 0.6991 / 0.6476 / 0.3523
V3R	0.6913 / 0.6991 / 0.6476 / 0.3523

4.2 路径级重构的 E2E 响应

V0 和 V2R 的平均 FPS 分别为 54.600 和 122.122，合并样本平均 E2E 延迟分别为 18.273 ms 和 8.140 ms。由此， $P_0 \rightarrow P_2$ 达到 $2.236671 \times$ 原始 FPS 比，按论文精度为 $2.24 \times$ ；平均延迟降低 55.4519%，按论文精度为 55.45%。整体结果见图 2。

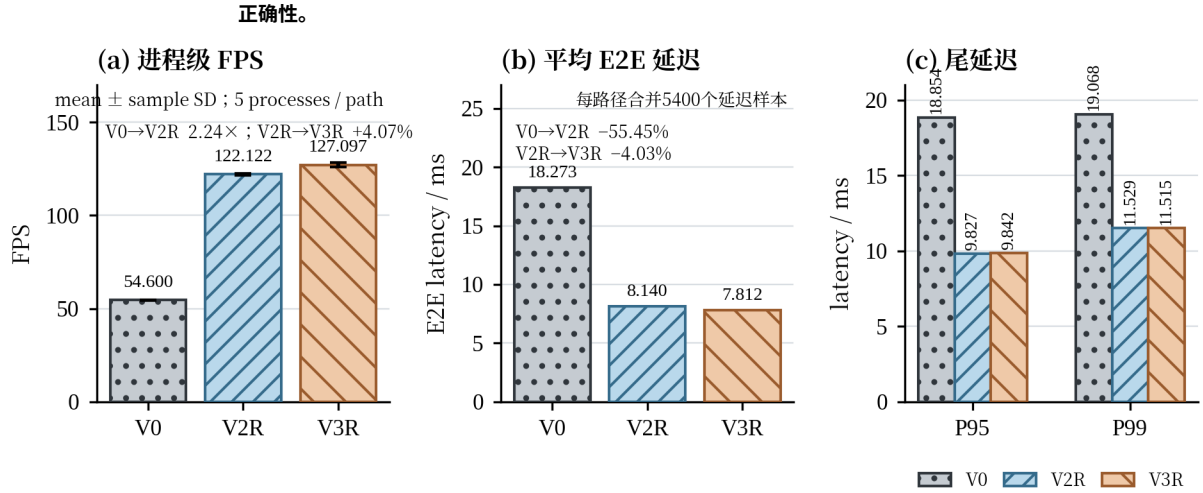


图 2 三条路径的端到端性能。(a) 为 5 个独立进程 FPS 的均值±样本标准差；(b)(c) 为每条路径合并 5400 个延迟样本的均值、P95 和 P99。

这项观测对应完整路径级干预：跨边界表示由 FP32 NCHW 张量变为 packed BGR uint8，输入张量形成位置由主机移至设备，并引入相应的 pageable 原始图像暂存组织，即 R, F, M 发生变化而 E 保持不变。式 (2) 给出的名义输入复制载荷由 4.9152 MB/frame 降至 0.1200 MB/frame，形成 $40.96 \times$ 结构对比；它是重构路径的一项派生属性，而非独立控制变量。由于未进行阶段插桩或总线/DRAM 流量测量， $2.24 \times$ E2E 响应只能归属于完整 $P_0 \rightarrow P_2$ 路径，不能归因于某个组件，也不能解释为 $40.96 \times$ 传输加速。

4.3 暂存策略的增量响应

在 R, F, E 、GPU 预处理语义、Engine、stream 和下游结构固定时，V3R 只把 M 由 pageable 改为 pinned。V3R 平均 FPS 为

127.097，平均 E2E 延迟为 7.812 ms；相对 V2R，FPS 提高 4.0738%（论文值+4.07%），平均延迟降低 4.0349%（论文值-4.03%）。与路径级重构相比，这一局部策略干预在受测系统中只形成有限的平均增量。该范围差异描述的是本实验的受控响应，不推出 pinned 内存在其他平台、模型或负载下的普遍收益。

4.4 平均性能与尾延迟响应

5 个独立进程中，V2R 与 V3R 的进程级 FPS 范围分别为 121.443–122.759 和 125.595–128.301，进程平均延迟范围分别为 8.098–8.185 ms 和 7.740–7.894 ms，分布见图 3。上述范围均不重叠，说明平均响应在 5 个独立进程中重复观察到，而非仅由单一异常进程形成；但不同路径的进程独立且不配对，本文不据此进行显著性推断。

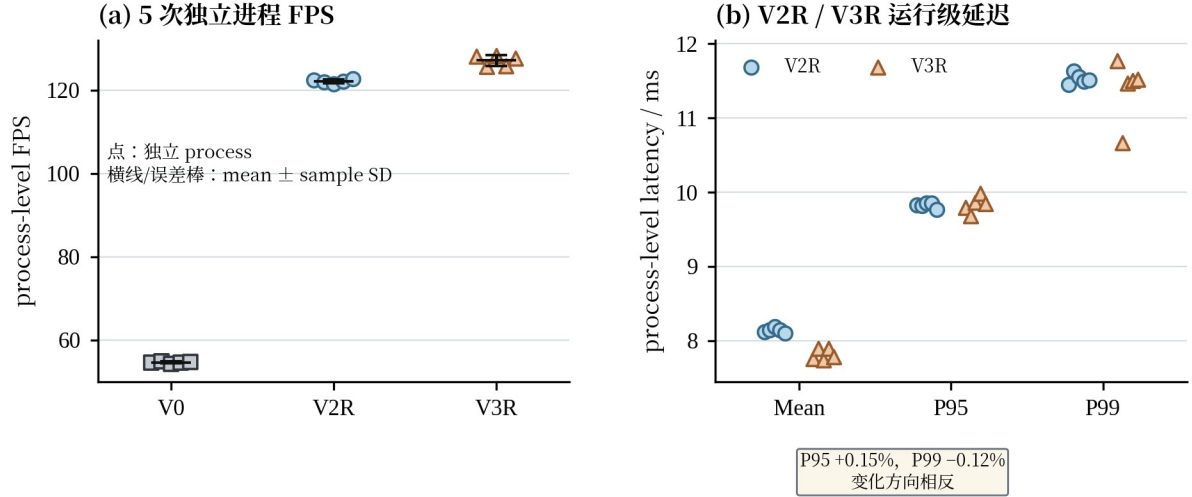


图3 运行级分布与尾延迟。各点为独立进程级描述量，横向偏移仅用于区分，不表示运行配对。

V3R 相对 V2R 的合并样本 P95 变化为 +0.1514%（论文值 +0.15%），P99 变化为 -0.1184%（论文值 -0.12%）；两个方向相反，未形成一致的尾延迟改善证据。这说明平均响应和尾延迟是不同评价维度：局部策略改变能够在本实验中改善均值，并不意味着 P95 与 P99 同步改善。该结果不支持“pinned 改善稳定性”或统计显著性结论。

4.5 解释边界与局限性

本文只评价单一 Jetson 平台、YOLOv8n TensorRT INT8 混合精度 Engine、NEU-DET 固定 180 幅离线回放工作负载和单帧顺序拓扑，未覆盖真实相机、跨平台、跨模型或跨数据集泛化。运行状态只有非连续温度观察，没有连续遥测、功耗或能耗测量，也没有独立归档的频率证据。实验未测量阶段级 H2D、预处理或 CUDA 核函数时间，也未测量实际总线或 DRAM 流量，因而不能进行组件级因果分配。每条路径 5 个独立进程仅支持描述性解释，不支持统计显著性推断。上述限制将结论约束为受测系统中的路径结构—E2E 响应关系。

5 结论

本文以固定推理对象下的输入数据路径为研究对象，将跨边界表示、输入张量形成位置、额外打包原始图像暂存策略和执行拓扑作为结构变量。在冻结 180 幅测试图像且三条路径任务级评价一致的条件，包含 R, F, M 变化且 E 保持不变的路径级 $P_0 \rightarrow P_2$ 使 FPS 达到 2.24×、平均 E2E 延迟降低 55.45%；仅改变 M 的局部 $P_2 \rightarrow P_3$ 使 FPS 提高 4.07%、平均延迟降低 4.03%，而

P95 增加 0.15%、P99 降低 0.12%，未形成一致的尾延迟改善证据。

结果说明，包含表示、张量形成位置及相应暂存组织变化的路径级干预，与仅改变主机暂存策略的局部干预属于不同层级，其完整 E2E 响应不能由单个 API 或名义载荷直接解释。对于本文这类固定推理对象的数据路径评价，应区分跨边界表示、张量形成位置和主机暂存策略，并将平均性能与尾延迟作为独立响应维度。该认识仅适用于本文单一 Jetson 平台、检测器/Engine、数据集、离线回放与单帧顺序配置，不作阶段级因果、统计显著性或跨平台/跨模型普适推断。

参考文献

- [1] Lv X, Duan F, Jiang J, 等. Deep Metallic Surface Defect Detection: The New Benchmark and Detection Network[J]. Sensors, 2020, 20 (6) : 1562. DOI:10.3390/s20061562.
- [2] Song K, Yan Y. A noise robust method based on completed local binary patterns for hot-rolled steel strip surface defects[J]. Applied Surface Science, 2013, 285: 858-864. DOI:10.1016/j.apsusc.2013.09.002.
- [3] Shao R, Zhou M, Li M, 等. TD-Net: tiny defect detection network for industrial products[J]. Complex & Intelligent Systems, 2024, 10: 3943-3954. DOI:10.1007/s40747-024-01362-x.
- [4] Chu Y, Yu X, Rong X. A Lightweight Strip Steel Surface Defect Detection Network Based on Improved YOLOv8[J]. Sensors, 2024, 24 (19) : 6495. DOI:10.3390/s24196495.
- [5] Zhang T, Pang H, Jiang C. GDM-YOLO: A Model for Steel Surface Defect Detection Based on YOLOv8s[J]. IEEE Access, 2024, 12: 148817-148825. DOI:10.1109/ACCESS.2024.3476908.
- [6] Ultralytics. Explore Ultralytics YOLOv8[EB/OL]. (2023) [2026-08-07]. <https://docs.ultralytics.com/models/yolov8/>.

- [7] Stäcker L, Fei J, Heidenreich P, 等. Deployment of Deep Neural Networks for Object Detection on Edge AI Devices With Runtime Optimization[C]//2021 IEEE/CVF International Conference on Computer Vision Workshops (ICCVW). IEEE, 2021: 1015-1022. DOI:10.1109/ICCVW54120.2021.00118.
- [8] Lee J, Han D, Kim J. PRESTO: Hybrid CPU-GPU Preprocessing Framework for Video-based AI Inference System[C]//Proceedings of the 23rd Annual International Conference on Mobile Systems, Applications and Services. ACM, 2025: 735-740. DOI:10.1145/3711875.3736688.
- [9] Weiss E, Caplan S, Horn K, 等. Real-Time Defect Detection in Electronic Components during Assembly through Deep Learning[J]. Electronics, 2024, 13 (8) : 1551. DOI:10.3390/electronics13081551.
- [10] Jacob B, Kligys S, Chen B, 等. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). 2018: 2704-2713.
- [11] Nagel M, Amjad R A, Van Baalen M, 等. Up or Down? Adaptive Rounding for Post-Training Quantization[C]//Proceedings of the 37th International Conference on Machine Learning: 卷 119. PMLR, 2020: 7197-7206.
- [12] NVIDIA Corporation. TensorRT 10.3 Release Notes[EB/OL]. (2024) [2026-08-07]. <https://docs.nvidia.com/deeplearning/tensorrt/archives/tensorrt-1030/pdf/TensorRT-Release-Notes.pdf>.
- [13] Kim N J, Lee J, Kim H. HyQ: Hardware-Friendly Post-Training Quantization for CNN-Transformer Hybrid Networks[C]//Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence, IJCAI-24. International Joint Conferences on Artificial Intelligence Organization, 2024: 4291-4299. DOI:10.24963/ijcai.2024/474.
- [14] Tang Y, Qian Y. High-speed railway track components inspection framework based on YOLOv8 with high-performance model deployment[J]. High-speed Railway, 2024, 2: 42-50. DOI:10.1016/j.hspr.2024.02.001.
- [15] NVIDIA Corporation. CUDA C++ Best Practices Guide: Release 12.6[EB/OL]. (2024) [2026-08-07]. <https://docs.nvidia.com/cuda/archive/12.6.0/cuda-c-best-practices-guide/index.html>.
- [16] Bateni S, Wang Z, Zhu Y, 等. Co-Optimizing Performance and Memory Footprint Via Integrated CPU/GPU Memory Management, an Implementation on Autonomous Driving Platform[C]//2020 IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS). IEEE, 2020: 310-323. DOI:10.1109/RTAS48715.2020.00007.
- [17] Rodriguez M, Yarza I, Kosmidis L, 等. Analysis of GPU Memory Allocation Characteristics[C]//16th Workshop on Parallel Programming and Run-Time Management Techniques for Many-Core Architectures and 14th Workshop on Design Tools and Architectures for Multicore Embedded Computing Platforms (PARMA-DITAM 2025): 卷 127. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025: 1:1-1:15. DOI:10.4230/OASICS.PARMA-DITAM.2025.1.
- [18] NVIDIA Corporation. CUDA C++ Programming Guide: Release 12.6[EB/OL]. (2024) [2026-08-07]. <https://docs.nvidia.com/cuda/archive/12.6.0/cuda-c-programming-guide/index.html>.
- [19] Kim S, Kim C, Kim S. Improving Performance of Real-Time Object Detection in Edge Device Through Concurrent Multi-Frame Processing[J]. IEEE Access, 2025, 13: 1522-1533. DOI:10.1109/ACCESS.2024.3520240.
- [20] Dean J, Barroso L A. The Tail at Scale[J]. Communications of the ACM, 2013, 56 (2) : 74-80. DOI:10.1145/2408776.2408794.
- [21] Shin D-J, Kim J-J. A Deep Learning Framework Performance Evaluation to Use YOLO in Nvidia Jetson Platform[J]. Applied Sciences, 2022, 12 (8) : 3734. DOI:10.3390/app12083734.
- [22] Lema D G, Sánchez-González L, Usamentiaga R, 等. Benchmarking deep learning models for surface defect detection: a reproducible and statistically-rigorous approach[J/OL]. Journal of Intelligent Manufacturing, 2025. DOI:10.1007/s10845-025-02672-8.